

The Complete C & C++ Programming Guide

From Zero to Mastery — Thinking, Building, and Engineering in C/C++

This document teaches you not just the syntax of C and C++, but how to *think* as a programmer. It covers the mental models, problem-solving frameworks, language mechanics, data structures, algorithms, memory management, and real-world engineering practices you need to write professional software.

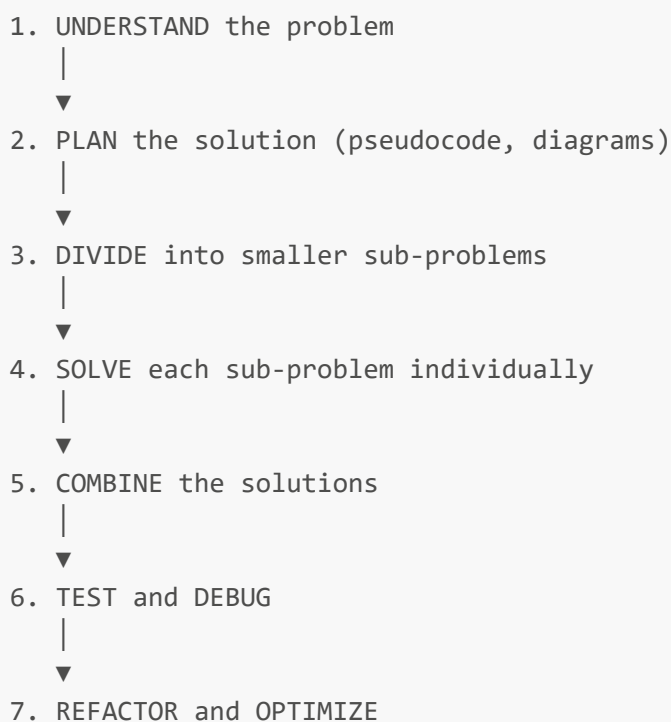
PART I: THE PROGRAMMER'S MINDSET

Chapter 1: How to Think Like a Programmer

Programming is not about memorizing syntax. It is about **decomposing problems** into smaller pieces that a machine can execute step by step.

The Problem-Solving Framework

Every programming task, from blinking an LED to building a search engine, follows the same mental process:



Step 1: Understand the Problem

Before writing a single line of code, you must be able to explain the problem in plain English. If you can't explain it, you can't code it.

Bad: "I need to sort numbers." **Good:** "I have an array of N integers. I need to rearrange them so that each element is less than or equal to the next element. The input can contain duplicates and negative numbers. N

can be up to 1 million."

Ask yourself:

- What are the **inputs**? (Types, ranges, edge cases)
- What are the **outputs**? (Format, expected behavior)
- What are the **constraints**? (Time limit, memory limit, real-time?)
- What are the **edge cases**? (Empty input, single element, all identical, maximum size)

Step 2: Plan Before You Code

Write pseudocode or draw a diagram BEFORE opening your editor. This is the most important step beginners skip.

Example Problem: "Find the second largest number in an array."

Pseudocode:

```
SET largest = first element
SET second_largest = negative infinity

FOR each element in the array:
    IF element > largest:
        second_largest = largest
        largest = element
    ELSE IF element > second_largest AND element != largest:
        second_largest = element

RETURN second_largest
```

Now translating to C is almost mechanical. The hard thinking is already done.

Step 3: Divide and Conquer

Complex problems are just collections of simple problems stacked together. If a problem feels overwhelming, break it into smaller pieces until each piece is trivially easy.

Example: "Build a command-line calculator"

Break it down:

1. Read a line of text from the user → `fgets()`
2. Parse the operator (+, -, *, /) → `if/switch` on a character
3. Parse the two numbers → `atof()` or `sscanf()`
4. Perform the operation → basic arithmetic
5. Print the result → `printf()`
6. Loop until the user types "quit" → `while` loop with `strcmp()`

Each of these is trivial on its own. Combined, they form a complete program.

The Debugging Mindset

When your code doesn't work (and it won't, at first), follow this process:

- 1. **Read the error message carefully.** Compilers tell you exactly what's wrong and on which line. Most beginners ignore error messages.
- 2. **Reproduce the bug.** Find the smallest input that triggers the problem.
- 3. **Form a hypothesis.** "I think the loop runs one too many times."
- 4. **Test the hypothesis.** Add `printf()` statements (or use a debugger) to verify.
- 5. **Fix and verify.** Make ONE change at a time. Test after each change.

Golden Rule: If you change 10 things at once and the bug goes away, you don't know which change fixed it. You've learned nothing, and the bug will come back.

How to Study Programming Effectively

Method	Effectiveness	Why
Watching tutorials passively	★☆☆☆☆	You feel like you're learning, but you're not. It's an illusion.
Copying code from a tutorial	★★☆☆☆	Slightly better. You practice typing. But you don't understand <i>why</i> .
Modifying tutorial code	★★★☆☆	Now you're experimenting. "What happens if I change this?"
Solving problems from scratch	★★★★☆	This is where real learning happens. The struggle IS the learning.
Teaching someone else	★★★★★	If you can explain it clearly, you truly understand it.

The #1 mistake beginners make: Reading for hours without writing code. Programming is a **craft**, like woodworking or playing guitar. You must practice with your hands, not just your eyes.

Chapter 2: Computational Thinking

Variables Are Boxes

A variable is a named box in memory that holds a value. The type determines the size of the box.

```
int age = 25;
```

Memory:

age

25

4 bytes (int)

Address: 0x7FFD

Name (label on the box)

Value (contents of the box)

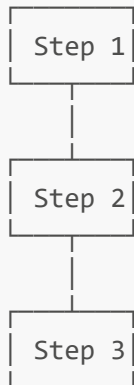
Type (size of the box)

Location (where the box is stored)

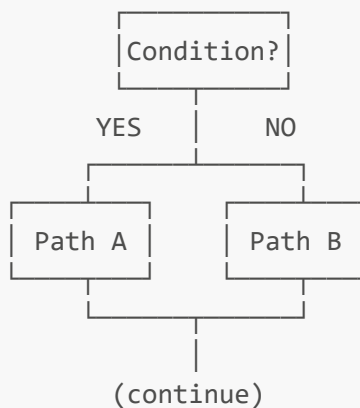
Control Flow Is a Flowchart

Every program is a combination of three structures:

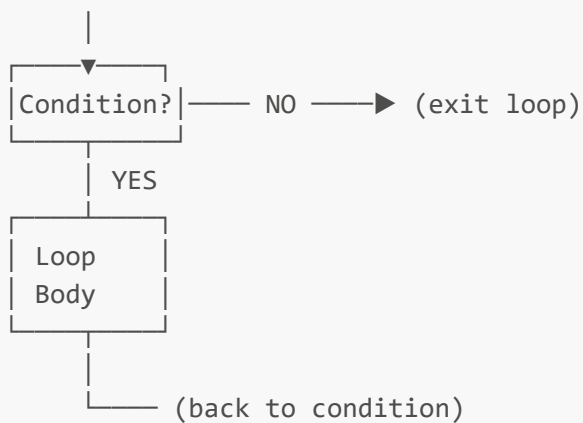
1. SEQUENCE (do this, then this, then this):



2. SELECTION (if this, do that; otherwise, do the other thing):



3. ITERATION (repeat until done):



Every algorithm ever written is built from these three building blocks. If you understand Sequence, Selection, and Iteration, you can understand any program.

Functions Are Machines

A function is a reusable machine: you put inputs in, it does work, and it gives you an output.

Inputs (Parameters)

↓

↑

Output (Return Value)

function body
(does work on the inputs)

```
int add(int a, int b) {  
    return a + b;  
}
```

← "Machine that adds two numbers"
← "Output: the sum"

```
int result = add(3, 5);
```

← "Feed 3 and 5 into the machine. Get 8."

Why functions matter:

- 1. **Reusability:** Write once, use everywhere.
- 2. **Abstraction:** You don't need to know HOW `printf()` works to USE it.
- 3. **Testability:** Small functions are easy to test in isolation.
- 4. **Readability:** `calculate_checksum(data, len)` is clearer than 20 lines of inline bit manipulation.

PART II: THE C LANGUAGE

Chapter 3: Data Types and Memory

Primitive Types

Type	Size (typical)	Range	Use
char	1 byte	-128 to 127 (or 0-255 unsigned)	Characters, small numbers, raw bytes
short	2 bytes	-32,768 to 32,767	Rarely used directly
int	4 bytes	-2.1 billion to 2.1 billion	General integer math
long	4 or 8 bytes	Platform-dependent	Avoid; use explicit types
long long	8 bytes	±9.2 quintillion	Large counters, timestamps
float	4 bytes	±3.4×10 ³⁸ (6-7 sig digits)	Approximate decimals
double	8 bytes	±1.7×10 ³⁰⁸ (15-16 sig digits)	Precise decimals

Fixed-Width Types (Use These in Embedded and Systems Code)

The standard types (`int`, `long`) have platform-dependent sizes. In embedded and systems programming, always use the explicit-width types from `<stdint.h>`:

```
#include <stdint.h>

uint8_t  byte_val;    // Exactly 8 bits, unsigned (0 to 255)
int8_t   sbyte_val;   // Exactly 8 bits, signed (-128 to 127)
uint16_t half_word;   // Exactly 16 bits, unsigned (0 to 65535)
int16_t  signed_hw;   // Exactly 16 bits, signed
uint32_t word;        // Exactly 32 bits, unsigned (0 to 4,294,967,295)
int32_t  signed_word; // Exactly 32 bits, signed
uint64_t dword;       // Exactly 64 bits, unsigned
int64_t  signed_dw;   // Exactly 64 bits, signed
```

How Numbers Are Stored in Memory

Unsigned integers are stored as pure binary:

```
uint8_t x = 42;

Binary: 0 0 1 0 1 0 1 0
Bit:    7 6 5 4 3 2 1 0

Value = 0×128 + 0×64 + 1×32 + 0×16 + 1×8 + 0×4 + 1×2 + 0×1
       = 32 + 8 + 2 = 42
```

Signed integers use **Two's Complement**. The most significant bit (MSB) is the sign bit:

```
int8_t x = -42;

Step 1: Start with +42:    0 0 1 0 1 0 1 0
Step 2: Flip all bits:     1 1 0 1 0 1 0 1
Step 3: Add 1:             1 1 0 1 0 1 1 0

Binary of -42: 11010110

Why two's complement? Because addition works naturally:
42 + (-42) = 00101010 + 11010110 = (1)00000000 = 0 ✓
(The carry-out bit is discarded)
```

Floating point (IEEE 754) stores numbers in scientific notation:

```
float x = -6.75;

In binary scientific notation: -1.1011 × 22
```

IEEE 754 Single Precision (32 bits):

S	Exponent	Mantissa
1	10000001	10110000000000000000000

1b

8 bits

23 bits

S = 1 (negative)

Exponent = 129 - 127 (bias) = 2

Mantissa = 1.1011 (implicit leading 1)

Value = $-1 \times 1.1011_2 \times 2^2 = -1 \times 1.6875 \times 4 = -6.75$

Why floating point is imprecise:

```
float a = 0.1f + 0.2f;
// a is NOT exactly 0.3!
// a = 0.30000001192092896...
// Because 0.1 cannot be represented exactly in base-2 floating point.

// NEVER compare floats with ==
if (a == 0.3f)           // WRONG: may fail due to rounding
if (fabs(a - 0.3f) < 0.0001f) // CORRECT: check if "close enough"
```

Endianness

Multi-byte values can be stored in memory two ways:

```
uint32_t x = 0x12345678;
```

Little-Endian (x86, ARM default):

Address: 0x00 0x01 0x02 0x03

Byte: 0x78 0x56 0x34 0x12 ← Least significant byte first

Big-Endian (Network byte order, some PowerPC):

Address: 0x00 0x01 0x02 0x03

Byte: 0x12 0x34 0x56 0x78 ← Most significant byte first

ARM Cortex-M is little-endian by default. This matters when you're reading multi-byte values from sensors or network packets.

Chapter 4: Operators (Complete Reference)

Arithmetic Operators

```
int a = 10, b = 3;
a + b    // 13 (addition)
```

```

a - b    // 7    (subtraction)
a * b    // 30   (multiplication)
a / b    // 3    (integer division – truncates, doesn't round!)
a % b    // 1    (modulo – remainder of division)

// WARNING: Integer division truncates toward zero
-7 / 2    // -3   (not -4)
-7 % 2    // -1

// For real division, cast to float:
(float)a / b // 3.333...

```

Comparison and Logical Operators

```

// Comparison (return 1 for true, 0 for false)
a == b    // Equal to
a != b    // Not equal to
a > b     // Greater than
a < b     // Less than
a >= b    // Greater than or equal
a <= b    // Less than or equal

// Logical (operate on boolean values)
x && y    // AND: true if BOTH are true
x || y    // OR: true if EITHER is true
!x        // NOT: inverts true/false

// SHORT-CIRCUIT EVALUATION:
// In (A && B), if A is false, B is NEVER evaluated
// In (A || B), if A is true, B is NEVER evaluated
// This is intentional and useful:
if (ptr != NULL && ptr->value > 0) // Safe: won't dereference NULL

```

Bitwise Operators (Critical for Embedded)

```

uint8_t a = 0b11001010; // 0xCA = 202
uint8_t b = 0b10110101; // 0xB5 = 181

a & b    // AND: 0b10000000 = 0x80 (both bits must be 1)
a | b    // OR: 0b11111111 = 0xFF (either bit can be 1)
a ^ b    // XOR: 0b01111111 = 0x7F (bits must be different)
~a       // NOT: 0b00110101 = 0x35 (flip all bits)
a << 2   // Left shift by 2: 0b00101000 (multiply by 4)
a >> 3   // Right shift by 3: 0b00011001 (divide by 8)

```

Common Bitwise Patterns:

```

// Set bit N
reg |= (1 << N);

// Clear bit N
reg &= ~(1 << N);

// Toggle bit N
reg ^= (1 << N);

// Check if bit N is set
if (reg & (1 << N)) { /* bit N is 1 */ }

// Extract bits [high:low] from a value
uint8_t field = (value >> low) & ((1 << (high - low + 1)) - 1);

// Example: Extract bits [7:4] from 0xAB
uint8_t upper_nibble = (0xAB >> 4) & 0x0F; // = 0x0A = 10

// Create a mask of N bits
uint32_t mask = (1U << N) - 1; // N=4 → 0b1111 = 0x0F

// Check if a number is a power of 2
if (x && !(x & (x - 1))) { /* x is a power of 2 */ }

// Multiply by power of 2 (fast)
x << 3; // x * 8

// Divide by power of 2 (fast, unsigned only)
x >> 3; // x / 8

// Swap two variables without a temporary (XOR swap)
a ^= b;
b ^= a;
a ^= b;

```

Operator Precedence (Memorize the Top 5)

Priority	Operators	Direction
1 (highest)	<code>() [] -> .</code>	Left to right
2	<code>! ~ ++ -- * & (type) sizeof</code>	Right to left
3	<code>* / %</code>	Left to right
4	<code>+ -</code>	Left to right
5	<code><< >></code>	Left to right
6	<code>< <= > >=</code>	Left to right
7	<code>== !=</code>	Left to right

Priority	Operators	Direction
8	<code>&</code> (bitwise AND)	Left to right
9	<code>^</code> (bitwise XOR)	Left to right
10	<code> </code> (bitwise OR)	Left to right
11	<code>&&</code> (logical AND)	Left to right
12	<code> </code> (logical OR)	Left to right
13	<code>?:</code> (ternary)	Right to left
14	<code>= += -=</code> etc.	Right to left
15 (lowest)	<code>,</code> (comma)	Left to right

The dangerous trap: Bitwise AND (`&`) has LOWER precedence than `==`.

```
if (reg & 0x04 == 0x04)    // WRONG! Parsed as: reg & (0x04 == 0x04) = reg & 1
if ((reg & 0x04) == 0x04) // CORRECT! Always parenthesize bitwise operations
```

Rule of thumb: When in doubt, add parentheses. Readability > cleverness.

Chapter 5: Control Flow

The `if / else if / else` Chain

```
if (temperature > 100) {
    printf("CRITICAL: Overheating!\n");
    shutdown_motor();
} else if (temperature > 80) {
    printf("WARNING: High temperature\n");
    reduce_speed();
} else if (temperature > 60) {
    printf("Normal operating range\n");
} else {
    printf("Cool\n");
}
```

Style rule: Always use braces `{}`, even for single-line bodies. This prevents a class of bugs where you add a second line thinking it's inside the `if`:

```
// BUG:
if (error)
    log_error();
    shutdown();    // THIS ALWAYS EXECUTES! It's not inside the if!
```

```
// SAFE:
if (error) {
    log_error();
    shutdown();    // Now it's clearly inside the if block
}
```

The `switch` Statement

```
switch (command) {
    case 'w':
        move_forward();
        break;           // MUST break, or execution "falls through" to next case
    case 's':
        move_backward();
        break;
    case 'a':
        turn_left();
        break;
    case 'd':
        turn_right();
        break;
    case 'q':
        printf("Quitting.\n");
        return 0;
    default:
        printf("Unknown command: %c\n", command);
        break;
}
```

The fall-through trap:

```
switch (x) {
    case 1:
        printf("One\n");
        // No break! Execution continues into case 2!
    case 2:
        printf("Two\n"); // This prints for BOTH x=1 and x=2
        break;
}
```

Loops

for loop — Use when you know how many iterations:

```
for (int i = 0; i < 10; i++) {
    printf("%d\n", i); // Prints 0 through 9
}
```

```
// Anatomy:
// for (initialization; condition; update)
//      |           |           |
//      |           |           |_____ Runs AFTER each iteration
//      |           |_____ Checked BEFORE each iteration
//      |_____ Runs ONCE before first iteration
```

while loop — Use when you don't know how many iterations:

```
while (uart_data_available()) {
    char c = uart_read();
    process_char(c);
}
// Condition is checked BEFORE the first iteration.
// If condition is false initially, the body never executes.
```

do...while loop — Use when the body must execute at least once:

```
do {
    printf("Enter a positive number: ");
    scanf("%d", &number);
} while (number <= 0); // Keep asking until valid input
```

Loop Control:

```
for (int i = 0; i < 100; i++) {
    if (i == 50) break; // EXIT the loop entirely
    if (i % 2 == 0) continue; // SKIP this iteration, go to next
    printf("%d\n", i); // Prints odd numbers: 1, 3, 5, ..., 49
}
```

Chapter 6: Pointers (The Most Important Chapter)

Pointers are THE concept that separates C from higher-level languages. Every embedded system, operating system, and game engine relies on pointers. Master this chapter and you master C.

What Is a Pointer?

A pointer is a variable that stores a **memory address** instead of a data value.

```
int x = 42;
int *p = &x; // p stores the ADDRESS of x
```

Memory Layout:

Address	Value	Variable	
0x1000	42	x	← "x contains the value 42"
0x1004	0x1000	p	← "p contains the ADDRESS of x"
*p	→ Follow the address → go to 0x1000 → find 42		
&x	→ "What is x's address?" → 0x1000		

The Two Key Operators

```
int x = 42;
int *p;      // Declare p as a "pointer to int"

p = &x;      // & (Address-of): Get the address of x, store it in p
printf("%d", *p); // * (Dereference): Follow the pointer to get the VALUE (42)

*p = 100;    // Change the value AT the address p points to
// Now x == 100 (we modified x through the pointer!)
```

Why Pointers Exist

Reason 1: Functions can modify caller's variables.

Without pointers, C passes arguments **by value** (copies):

```
void broken_swap(int a, int b) {
    int temp = a;
    a = b;    // Modifies LOCAL COPY only
    b = temp; // Modifies LOCAL COPY only
}
// x and y are UNCHANGED after calling broken_swap(x, y)

void working_swap(int *a, int *b) {
    int temp = *a;
    *a = *b;    // Modifies the ORIGINAL variable through pointer
    *b = temp;
}
// Call with: working_swap(&x, &y)
// x and y are ACTUALLY SWAPPED
```

Reason 2: Efficiently pass large data.

```
// BAD: Copies the entire 1000-int array onto the stack (4000 bytes!)
void process_bad(int data[1000]) { ... }
```

```
// GOOD: Passes only a pointer (4 or 8 bytes)
void process_good(int *data, int length) { ... }
```

Reason 3: Dynamic memory allocation.

```
// Allocate an array whose size is determined at runtime
int n;
scanf("%d", &n);
int *arr = (int *)malloc(n * sizeof(int));
// arr points to a block of n integers on the heap
// Use arr[0], arr[1], ..., arr[n-1]
free(arr); // MUST free when done, or memory leaks
```

Reason 4: Hardware register access (embedded).

```
// GPIOA's Output Data Register is at physical address 0x40020014
volatile uint32_t *odr = (volatile uint32_t *)0x40020014;
*odr |= (1 << 5); // Turn on LED by writing to hardware
```

Pointer Arithmetic

When you add to a pointer, it advances by the SIZE of the pointed-to type:

```
int arr[] = {10, 20, 30, 40, 50};
int *p = arr; // p points to arr[0]

p + 1 // Points to arr[1] (address + 4 bytes, because sizeof(int) = 4)
p + 2 // Points to arr[2] (address + 8 bytes)
*(p + 3) // Value at arr[3] = 40

// In fact, arr[i] is EXACTLY equivalent to *(arr + i)
// And &arr[i] is EXACTLY equivalent to (arr + i)
```

Memory:

Address:	0x1000	0x1004	0x1008	0x100C	0x1010
Value:	10	20	30	40	50
Index:	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]

▲
|
p (when p = arr)

p + 2: —————▶ (moved 2 × sizeof(int) = 8 bytes)

▲
Points to arr[2] = 30

Arrays and Pointers

In C, an array name **decays** into a pointer to its first element in most contexts:

```
int arr[5] = {10, 20, 30, 40, 50};

arr          // Type: int* (pointer to first element)
&arr[0]      // Same thing: pointer to first element
&arr         // Type: int(*)[5] (pointer to the ENTIRE array – different type!)

sizeof(arr)   // 20 (5 × 4 bytes – only works in the scope where arr is
              // declared)
sizeof(&arr[0]) // 4 or 8 (size of a POINTER, not the array)
```

Strings in C

C has no string type. A "string" is just a **char** array terminated by a null character ('\0'):

```
char name[] = "Hello";

// Memory:
// name[0] = 'H' (0x48)
// name[1] = 'e' (0x65)
// name[2] = 'l' (0x6C)
// name[3] = 'l' (0x6C)
// name[4] = 'o' (0x6F)
// name[5] = '\0' (0x00) ← Null terminator! sizeof(name) = 6, strlen(name) = 5

char *str = "World";
// str is a pointer to a string literal in read-only memory.
// str[0] = 'W' (you can READ it)
// str[0] = 'X' → UNDEFINED BEHAVIOR (you CANNOT write to string literals!)
```

Common string functions (`#include <string.h>`):

```
strlen(s)          // Length (not counting '\0')
strcmp(a, b)        // Compare: returns 0 if equal, <0 if a<b, >0 if a>b
strncmp(a, b, n)    // Compare first n characters
strcpy(dst, src)     // Copy src to dst (DANGEROUS: no bounds check!)
strncpy(dst, src, n) // Copy at most n characters (safer)
strcat(dst, src)     // Append src to end of dst
strncat(dst, src, n) // Append at most n characters
strstr(hay, needle)  // Find substring; returns pointer or NULL
strchr(s, c)         // Find first occurrence of character c
```

```
memcpy(dst, src, n) // Copy n bytes (for any data, not just strings)
memset(dst, val, n) // Fill n bytes with value val (often 0)
```

Pointer to Pointer (Double Pointer)

```
int x = 42;
int *p = &x;    // p points to x
int **pp = &p;   // pp points to p

**pp = 100;      // Follow pp → get p → follow p → get x → set x = 100
// Now x == 100
```

Primary use: Modifying a pointer inside a function:

```
void allocate(int **out_ptr, int size) {
    *out_ptr = (int *)malloc(size * sizeof(int));
}

int *arr = NULL;
allocate(&arr, 10); // Now arr points to allocated memory
```

Function Pointers

Functions live in memory at specific addresses. You can store that address in a pointer and call the function through it:

```
// Declare a function pointer type
typedef int (*MathFunc)(int, int);

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

MathFunc operation;
operation = add;    // Point to the add function
int result = operation(3, 4); // Calls add(3, 4) = 7

operation = mul;
result = operation(3, 4);    // Calls mul(3, 4) = 12
```

Practical use — Callback pattern:

```
// A generic sort function that accepts a comparison function
void sort(int *arr, int n, int (*compare)(int, int)) {
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
            if (compare(arr[j], arr[j+1]) > 0) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

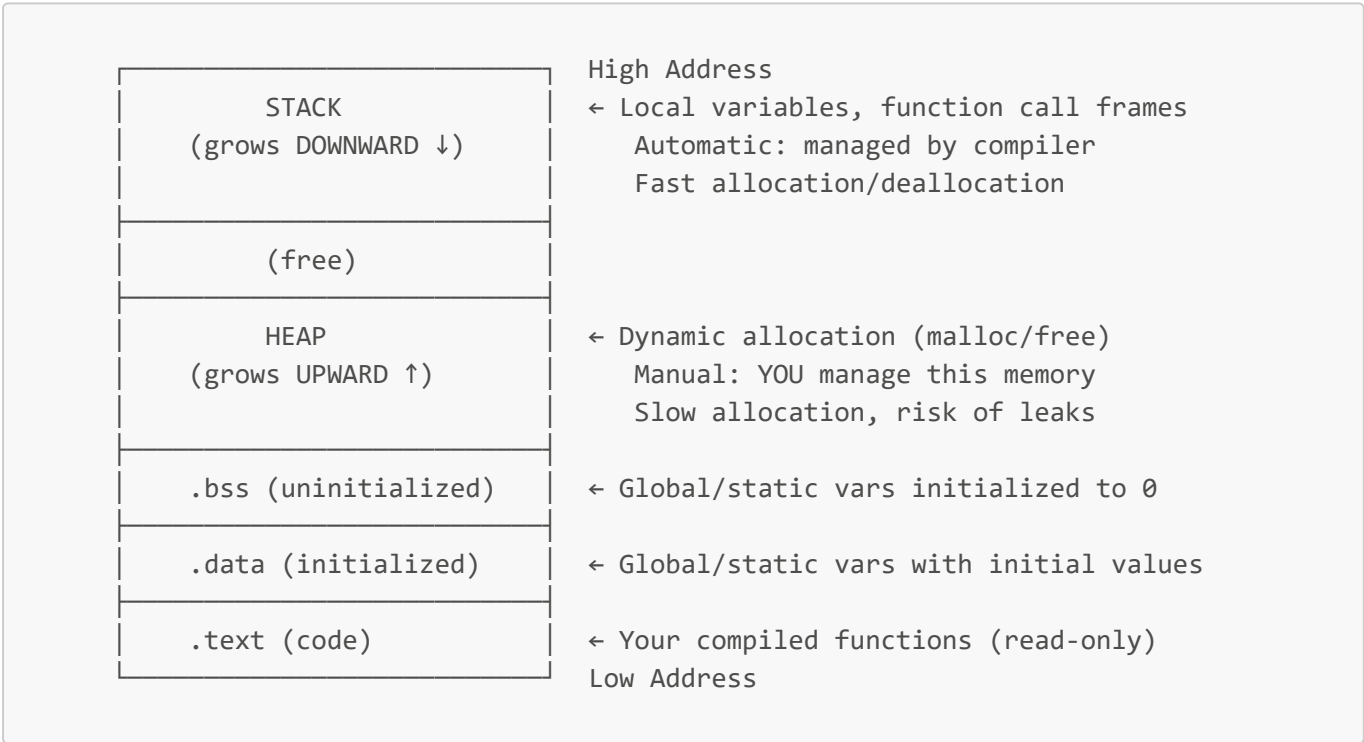
int ascending(int a, int b) { return a - b; }
int descending(int a, int b) { return b - a; }

sort(arr, n, ascending); // Sort smallest to largest
sort(arr, n, descending); // Sort largest to smallest
```

This is exactly how `qsort()` in the C standard library works. Your ZeroHAL task scheduler uses function pointers too (`void (*task_func)(void)`).

Chapter 7: Memory Management

The Four Memory Regions



Stack vs Heap

Property	Stack	Heap
Speed	Very fast (just move SP)	Slower (search for free block)
Management	Automatic (compiler does it)	Manual (malloc/free)

Property	Stack	Heap
Size	Small (1-8 MB typical)	Large (limited by RAM)
Lifetime	Until function returns	Until you call <code>free()</code>
Risk	Stack overflow (deep recursion)	Memory leaks, use-after-free
Thread safety	Each thread gets its own stack	Shared between threads (needs locking)

Dynamic Memory Allocation

```
#include <stdlib.h>

// Allocate memory for n integers
int *arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) {
    // malloc FAILED – out of memory
    fprintf(stderr, "Error: malloc failed\n");
    return -1;
}

// Use the memory
for (int i = 0; i < n; i++) {
    arr[i] = i * 10;
}

// MUST free when done – otherwise MEMORY LEAK
free(arr);
arr = NULL; // Good practice: prevent dangling pointer

// calloc: Like malloc but zeroes the memory
int *zeros = (int *)calloc(n, sizeof(int)); // All elements are 0

// realloc: Resize an existing allocation
arr = (int *)realloc(arr, new_size * sizeof(int));
// May move the block to a new address! Old pointer is invalidated.
```

The Seven Deadly Memory Sins

```
// 1. MEMORY LEAK – Forgetting to free
void leak() {
    int *p = malloc(100);
    return; // p is lost! The 100 bytes can never be freed.
}

// 2. USE-AFTER-FREE – Accessing freed memory
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p); // UNDEFINED BEHAVIOR! Memory may be reused.
```

```
// 3. DOUBLE FREE – Freeing the same memory twice
free(p);
free(p); // CRASH or heap corruption!

// 4. BUFFER OVERFLOW – Writing past array bounds
char buf[10];
strcpy(buf, "This string is way too long for the buffer");
// Overwrites adjacent memory → crashes, security vulnerabilities

// 5. DANGLING POINTER – Pointer to local variable that no longer exists
int *get_local() {
    int x = 42;
    return &x; // x is destroyed when function returns!
} // The returned pointer is INVALID.

// 6. NULL DEREFERENCE – Forgetting to check malloc's return
int *p = malloc(1000000000); // Might fail
*p = 42; // If malloc returned NULL → CRASH (segfault)

// 7. UNINITIALIZED MEMORY – Reading before writing
int *p = malloc(sizeof(int));
printf("%d\n", *p); // UNDEFINED: could be 0, garbage, or anything
```

Chapter 8: Structs, Enums, Unions, and Typedefs

Structs

A struct groups related variables into a single composite type:

```
typedef struct {
    char    name[32];
    uint8_t age;
    float   gpa;
    uint32_t student_id;
} Student;

Student s;
s.age = 20;
strcpy(s.name, "Dhruv");

// Access through a pointer (very common in embedded):
Student *ptr = &s;
ptr->age = 21; // Arrow operator: equivalent to (*ptr).age
printf("%s\n", ptr->name);
```

Struct Memory Layout and Padding:

The compiler may insert padding bytes between struct members for alignment:

```
typedef struct {
    char    a;      // 1 byte
    // 3 bytes padding (to align next int to 4-byte boundary)
    int     b;      // 4 bytes
    char    c;      // 1 byte
    // 3 bytes padding (to make struct size a multiple of 4)
} Padded;          // sizeof = 12 (not 6!)

typedef struct {
    int     b;      // 4 bytes
    char    a;      // 1 byte
    char    c;      // 1 byte
    // 2 bytes padding
} Reordered;       // sizeof = 8 (better!)
```

Rule: Order struct members from largest to smallest to minimize padding.

In embedded, you can pack structs tightly:

```
typedef struct __attribute__((packed)) {
    char a;      // 1 byte
    int  b;      // 4 bytes (no padding!)
    char c;      // 1 byte
} Packed;       // sizeof = 6

// WARNING: Packed structs may cause unaligned access faults on some ARM chips
// Use only when matching a wire protocol or hardware register layout
```

Enums

Enums create named integer constants:

```
typedef enum {
    STATE_IDLE = 0,
    STATE_RUNNING,    // = 1 (auto-increments)
    STATE_ERROR,      // = 2
    STATE_SHUTDOWN    // = 3
} SystemState;

SystemState current = STATE_IDLE;

switch (current) {
    case STATE_IDLE:    start_motor(); break;
    case STATE_RUNNING: check_temp(); break;
    case STATE_ERROR:   halt_motor(); break;
    case STATE_SHUTDOWN: power_off(); break;
}
```

Unions

A union stores multiple types in the SAME memory location. Only one member is valid at a time:

```
typedef union {
    uint32_t word;           // Access as a single 32-bit word
    uint8_t bytes[4];       // Access as 4 individual bytes
    struct {
        uint16_t low;       // Access as two 16-bit halves
        uint16_t high;
    } halves;
} DataView;

DataView d;
d.word = 0xDEADBEEF;
printf("Byte 0: 0x%02X\n", d.bytes[0]); // 0xEF (little-endian)
printf("Low half: 0x%04X\n", d.halves.low); // 0xBEEF
```

Practical use: Parsing binary network packets or hardware register fields without bit-shifting.

Chapter 9: The Preprocessor

The C preprocessor runs BEFORE the compiler. It performs text substitution.

Macros

```
// Simple constant (preferred over magic numbers)
#define MAX_BUFFER_SIZE 256
#define PI 3.14159265358979

// Function-like macro (inline, no function call overhead)
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#define ABS(x) ((x) < 0 ? -(x) : (x))

// ALWAYS parenthesize macro arguments to prevent precedence bugs:
#define BAD_SQUARE(x) x * x
#define GOOD_SQUARE(x) ((x) * (x))

BAD_SQUARE(3 + 1) // Expands to: 3 + 1 * 3 + 1 = 7 (WRONG!)
GOOD_SQUARE(3 + 1) // Expands to: ((3 + 1) * (3 + 1)) = 16 (CORRECT!)

// WARNING: Macros with side effects are dangerous:
GOOD_SQUARE(i++) // Expands to: ((i++) * (i++)) → i is incremented TWICE!
```

Conditional Compilation

```

// Include guards (prevent double-inclusion of headers)
#ifndef MY_HEADER_H
#define MY_HEADER_H

// ... header contents ...

#endif // MY_HEADER_H

// Platform-specific code
#ifdef _WIN32
#include <windows.h>
#elif defined(__linux__)
#include <unistd.h>
#elif defined(__APPLE__)
#include <mach/mach.h>
#endif

// Debug logging (compiled out in release builds)
#ifdef DEBUG
#define LOG(fmt, ...) printf("[DEBUG] " fmt "\n", ##__VA_ARGS__)
#else
#define LOG(fmt, ...) ((void)0) // Compiles to nothing
#endif

```

Useful Preprocessor Tricks

```

// Stringify: Turn a token into a string literal
#define STR(x) #x
printf(STR(hello)); // Prints: hello

// Token pasting: Concatenate tokens
#define REGISTER(name) GPIO##name##_BASE
REGISTER(A) // Expands to: GPIOA_BASE

// Compile-time assertion (C11)
_Static_assert(sizeof(int) == 4, "int must be 4 bytes");

// Built-in macros
__FILE__ // Current source file name (string)
__LINE__ // Current line number (integer)
__func__ // Current function name (string, C99)
__DATE__ // Compilation date
__TIME__ // Compilation time

```

Chapter 10: Data Structures

Arrays

```
// Static array (size known at compile time, lives on stack)
int scores[100];

// Variable-length array (C99, lives on stack – risky for large sizes!)
int n = get_input();
int dynamic[n]; // Can cause stack overflow if n is large

// 2D array
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
// matrix[row][col]
// In memory: stored row by row (row-major order)
```

Linked List

```
typedef struct Node {
    int data;
    struct Node *next; // Pointer to next node (self-referential)
} Node;

// Create a new node
Node *create_node(int value) {
    Node *n = (Node *)malloc(sizeof(Node));
    n->data = value;
    n->next = NULL;
    return n;
}

// Insert at the beginning (O(1))
void push_front(Node **head, int value) {
    Node *new_node = create_node(value);
    new_node->next = *head;
    *head = new_node;
}

// Print the list
void print_list(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

// Free the entire list
void free_list(Node **head) {
```

```

Node *current = *head;
while (current != NULL) {
    Node *next = current->next;
    free(current);
    current = next;
}
*head = NULL;
}

```

When to use arrays vs linked lists:

Operation	Array	Linked List
Access by index	O(1) — instant	O(n) — must traverse
Insert at beginning	O(n) — shift everything	O(1) — just update pointers
Insert at end	O(1) amortized	O(n) — must find end (O(1) with tail pointer)
Search	O(n) unsorted, O(log n) sorted	O(n) always
Memory	Contiguous, cache-friendly	Scattered, cache-unfriendly
Size	Fixed (or must realloc)	Grows dynamically

Stack (LIFO — Last In, First Out)

```

#define STACK_MAX 256

typedef struct {
    int data[STACK_MAX];
    int top;
} Stack;

void stack_init(Stack *s)    { s->top = -1; }
int  stack_empty(Stack *s)   { return s->top == -1; }
int  stack_full(Stack *s)    { return s->top == STACK_MAX - 1; }

void stack_push(Stack *s, int val) {
    if (!stack_full(s)) s->data[++(s->top)] = val;
}

int stack_pop(Stack *s) {
    if (!stack_empty(s)) return s->data[(s->top)--];
    return -1; // Error
}

int stack_peek(Stack *s) {
    if (!stack_empty(s)) return s->data[s->top];
    return -1;
}

```

Uses: Function call stack, expression evaluation, undo/redo, parsing (balanced brackets).

Queue (FIFO — First In, First Out)

```
#define QUEUE_SIZE 64

typedef struct {
    int data[QUEUE_SIZE];
    int head;
    int tail;
    int count;
} Queue;

void queue_init(Queue *q) { q->head = 0; q->tail = 0; q->count = 0; }
int queue_empty(Queue *q) { return q->count == 0; }
int queue_full(Queue *q) { return q->count == QUEUE_SIZE; }

void queue_enqueue(Queue *q, int val) {
    if (!queue_full(q)) {
        q->data[q->tail] = val;
        q->tail = (q->tail + 1) % QUEUE_SIZE; // Circular wrap
        q->count++;
    }
}

int queue_dequeue(Queue *q) {
    if (!queue_empty(q)) {
        int val = q->data[q->head];
        q->head = (q->head + 1) % QUEUE_SIZE;
        q->count--;
        return val;
    }
    return -1;
}
```

Uses: UART ring buffers (exactly like your ZeroHAL!), task scheduling, BFS graph traversal, message passing between threads.

Hash Table (Dictionary)

A hash table maps keys to values with $O(1)$ average lookup time.

```
#define TABLE_SIZE 256

typedef struct Entry {
    char *key;
    int value;
    struct Entry *next; // Chaining for collision resolution
} Entry;
```

```

typedef struct {
    Entry *buckets[TABLE_SIZE];
} HashTable;

// Simple hash function (djb2 by Dan Bernstein)
unsigned int hash(const char *key) {
    unsigned int h = 5381;
    while (*key) {
        h = ((h << 5) + h) + *key; // h * 33 + c
        key++;
    }
    return h % TABLE_SIZE;
}

void ht_set(HashTable *ht, const char *key, int value) {
    unsigned int idx = hash(key);
    Entry *e = ht->buckets[idx];

    // Check if key already exists
    while (e) {
        if (strcmp(e->key, key) == 0) {
            e->value = value; // Update existing
            return;
        }
        e = e->next;
    }

    // Insert new entry at head of chain
    Entry *new_entry = (Entry *)malloc(sizeof(Entry));
    new_entry->key = strdup(key);
    new_entry->value = value;
    new_entry->next = ht->buckets[idx];
    ht->buckets[idx] = new_entry;
}

int ht_get(HashTable *ht, const char *key, int *out_value) {
    unsigned int idx = hash(key);
    Entry *e = ht->buckets[idx];
    while (e) {
        if (strcmp(e->key, key) == 0) {
            *out_value = e->value;
            return 1; // Found
        }
        e = e->next;
    }
    return 0; // Not found
}

```

Chapter 11: Algorithms

Sorting

Bubble Sort — Simple but slow. $O(n^2)$. Good for understanding, bad for production.

```
void bubble_sort(int *arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Merge Sort — Efficient, stable, $O(n \log n)$. Divide and conquer.

```
void merge(int *arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *L = malloc(n1 * sizeof(int));
    int *R = malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    free(L); free(R);
}

void merge_sort(int *arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Searching

Binary Search — $O(\log n)$. Array MUST be sorted.

```

int binary_search(int *arr, int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Avoids overflow vs (low+high)/2
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Not found
}

```

Big-O Notation (Algorithm Complexity)

Big-O describes how an algorithm's time or space grows relative to input size:

Notation	Name	Example	1,000 elements	1,000,000 elements
O(1)	Constant	Array access by index	1 op	1 op
O(log n)	Logarithmic	Binary search	10 ops	20 ops
O(n)	Linear	Linear search	1,000 ops	1,000,000 ops
O(n log n)	Linearithmic	Merge sort, Quick sort	10,000 ops	20,000,000 ops
O(n ²)	Quadratic	Bubble sort, Selection sort	1,000,000 ops	10 ¹² ops (too slow!)
O(2 ⁿ)	Exponential	Brute-force all subsets	10 ³⁰⁰ (impossible)	—

Chapter 12: File I/O

```

#include <stdio.h>

// Writing to a file
FILE *fp = fopen("data.txt", "w"); // "w" = write (creates/overwrites)
if (fp == NULL) {
    perror("fopen failed"); // Prints system error message
    return -1;
}
fprintf(fp, "Temperature: %.2f°C\n", 23.45);
fprintf(fp, "Humidity: %d%%\n", 65);
fclose(fp); // ALWAYS close files

// Reading from a file
fp = fopen("data.txt", "r"); // "r" = read
char line[256];
while (fgets(line, sizeof(line), fp) != NULL) {
    printf("Read: %s", line);
}
fclose(fp);

```

```
// Binary file I/O (for raw data: structs, sensor logs)
typedef struct { float temp; uint32_t timestamp; } Record;

Record rec = {23.45f, 1696300000};

fp = fopen("log.bin", "wb"); // "wb" = write binary
fwrite(&rec, sizeof(Record), 1, fp);
fclose(fp);

fp = fopen("log.bin", "rb"); // "rb" = read binary
Record loaded;
fread(&loaded, sizeof(Record), 1, fp);
printf("Temp: %.2f at %u\n", loaded.temp, loaded.timestamp);
fclose(fp);
```

File open modes:

Mode	Meaning
"r"	Read (file must exist)
"w"	Write (creates new / truncates existing)
"a"	Append (creates new / writes to end of existing)
"r+"	Read and write (file must exist)
"w+"	Read and write (creates new / truncates existing)
"rb" / "wb"	Binary mode (no newline translation on Windows)

PART III: THE C++ LANGUAGE

Chapter 13: C++ — What It Adds to C

C++ is a superset of C. Almost all valid C code is valid C++. But C++ adds powerful features for building larger, more complex systems.

Key Additions Over C

Feature	C	C++
Classes / Objects	Structs only (data, no methods)	Full OOP (data + methods + inheritance)
Function overloading	No	Yes (same name, different parameters)
Operator overloading	No	Yes (define what +, ==, << mean for your types)
References	No (only pointers)	Yes (<code>int &ref = x;</code>)

Feature	C	C++
Templates	No	Yes (generic programming)
Exceptions	No (use error codes)	Yes (<code>try/catch/throw</code>)
Namespaces	No	Yes (<code>std::</code> , <code>my_lib::</code>)
STL (Standard Template Library)	No	Yes (vectors, maps, algorithms)
RAII / Destructors	No (manual cleanup)	Yes (automatic cleanup)
<code>auto</code> type inference	No	Yes (C++11)
Lambda functions	No	Yes (C++11)
Smart pointers	No	Yes (C++11: <code>unique_ptr</code> , <code>shared_ptr</code>)

Hello World in C++

```
#include <iostream> // C++ I/O (replaces stdio.h)
#include <string>    // C++ string class (replaces char arrays)

int main() {
    std::string name;
    std::cout << "Enter your name: ";
    std::cin >> name;
    std::cout << "Hello, " << name << "!" << std::endl;
    return 0;
}
```

References (Safer Pointers)

```
int x = 42;
int &ref = x;    // ref IS x. Not a copy, not a pointer. It IS the same variable.

ref = 100;        // x is now 100
printf("%d\n", x); // Prints 100

// Use in function parameters:
void increment(int &value) { // Takes a reference, not a copy
    value++; // Modifies the original variable directly
}

int num = 10;
increment(num); // num is now 11 (no & needed at call site)
```

When to use references vs pointers:

- Use **references** when the argument will always exist (never NULL) and you don't need to change what it points to.
- Use **pointers** when the argument might be NULL, or you need pointer arithmetic, or you need to reassign it.

Chapter 14: Object-Oriented Programming (OOP)

Classes

A class bundles data (member variables) and behavior (member functions/methods) together:

```
class Motor {  
private:    // Only accessible inside this class  
    int speed;  
    bool running;  
  
public:    // Accessible from outside  
    // Constructor: called when object is created  
    Motor() : speed(0), running(false) {  
        std::cout << "Motor created\n";  
    }  
  
    // Destructor: called when object is destroyed (RAII)  
    ~Motor() {  
        if (running) stop();  
        std::cout << "Motor destroyed\n";  
    }  
  
    void start() {  
        running = true;  
        std::cout << "Motor started\n";  
    }  
  
    void stop() {  
        running = false;  
        speed = 0;  
        std::cout << "Motor stopped\n";  
    }  
  
    void set_speed(int new_speed) {  
        if (new_speed >= 0 && new_speed <= 100) {  
            speed = new_speed;  
        }  
    }  
  
    int get_speed() const { // 'const' means this method doesn't modify the  
object  
        return speed;  
    }  
};
```

```
// Usage:
Motor m;           // Constructor runs: "Motor created"
m.start();         // "Motor started"
m.set_speed(75);
printf("Speed: %d\n", m.get_speed()); // 75
// When m goes out of scope, destructor runs: "Motor stopped", "Motor destroyed"
```

The Four Pillars of OOP

1. Encapsulation — Hide internal data, expose a clean interface.

```
class BankAccount {
private:
    double balance; // Can't be accessed directly from outside
public:
    void deposit(double amount) {
        if (amount > 0) balance += amount; // Validation!
    }
    double get_balance() const { return balance; }
    // No "set_balance()" - you can't arbitrarily change the balance
};
```

2. Inheritance — Create new classes based on existing ones.

```
class Sensor {
protected: // Accessible by derived classes, but not by outsiders
    uint8_t i2c_address;
public:
    Sensor(uint8_t addr) : i2c_address(addr) {}
    virtual float read() = 0; // Pure virtual: MUST be implemented by derived
class
    virtual ~Sensor() {} // Virtual destructor for proper cleanup
};

class TemperatureSensor : public Sensor {
public:
    TemperatureSensor(uint8_t addr) : Sensor(addr) {}

    float read() override { // Implements the pure virtual function
        // Read I2C register and convert to Celsius
        uint8_t raw = i2c_read_reg(i2c_address, 0x00);
        return raw * 0.0625f;
    }
};

class PressureSensor : public Sensor {
public:
    PressureSensor(uint8_t addr) : Sensor(addr) {}
```

```

float read() override {
    uint16_t raw = i2c_read_reg16(i2c_address, 0xF7);
    return raw / 256.0f;
}
};

```

3. Polymorphism — Treat different objects through the same interface.

```

void print_reading(Sensor &s) { // Works with ANY Sensor subclass
    std::cout << "Reading: " << s.read() << std::endl;
}

TemperatureSensor temp(0x48);
PressureSensor press(0x77);

print_reading(temp); // Calls TemperatureSensor::read()
print_reading(press); // Calls PressureSensor::read()
// Same function, different behavior – that's polymorphism!

```

4. Abstraction — Show only essential details, hide complexity. The `Sensor` base class is an abstraction. Users call `read()` without knowing whether it uses I2C, SPI, ADC, or a crystal ball internally.

Chapter 15: Templates (Generic Programming)

Templates let you write code that works with ANY type:

```

// Function template: works with int, float, double, or any comparable type
template <typename T>
T find_max(T a, T b) {
    return (a > b) ? a : b;
}

int    x = find_max(3, 7);           // T = int, returns 7
double y = find_max(3.14, 2.71);    // T = double, returns 3.14
char   z = find_max('a', 'z');      // T = char, returns 'z'

// Class template
template <typename T, int SIZE>
class FixedArray {
private:
    T data[SIZE];
    int count;
public:
    FixedArray() : count(0) {}

    bool push(const T &item) {
        if (count < SIZE) {
            data[count++] = item;
        }
    }
};

```

```

        return true;
    }
    return false;
}

T &operator[](int index) { return data[index]; }
int size() const { return count; }
};

FixedArray<int, 10> numbers;      // Array of 10 ints
FixedArray<float, 100> readings; // Array of 100 floats

```

Chapter 16: The Standard Template Library (STL)

The STL provides battle-tested, high-performance data structures and algorithms.

Containers

```

#include <vector>
#include <string>
#include <map>
#include <set>
#include <queue>
#include <stack>
#include <unordered_map>
#include <algorithm>

// — vector (dynamic array) —
std::vector<int> nums = {5, 3, 8, 1, 9};
nums.push_back(42);      // Add to end
nums.pop_back();         // Remove from end
nums[0] = 10;            // Access by index
nums.size();             // Number of elements
nums.empty();            // true if empty

for (int n : nums) {     // Range-based for loop (C++11)
    std::cout << n << " ";
}

// — string —
std::string s = "Hello";
s += " World";          // Concatenation
s.length();             // 11
s.substr(0, 5);          // "Hello"
s.find("World");         // 6 (position)
s.find("xyz");           // std::string::npos (not found)

// — map (sorted key-value pairs, O(log n) lookup) —
std::map<std::string, int> ages;
ages["Alice"] = 25;

```

```

ages["Bob"] = 30;
ages.count("Alice");      // 1 (exists)
ages.count("Charlie");    // 0 (doesn't exist)

for (auto &[name, age] : ages) { // Structured bindings (C++17)
    std::cout << name << " is " << age << "\n";
}

// — unordered_map (hash table, O(1) average lookup) —
std::unordered_map<std::string, int> fast_map;
fast_map["key"] = 42;    // Same interface as map, but faster

// — set (unique sorted elements) —
std::set<int> unique_nums = {5, 3, 8, 3, 5, 1};
// Contains: {1, 3, 5, 8} – duplicates removed, sorted

// — stack and queue —
std::stack<int> stk;
stk.push(10); stk.push(20);
stk.top();    // 20
stk.pop();    // Remove 20

std::queue<int> q;
q.push(10); q.push(20);
q.front();   // 10
q.pop();     // Remove 10

// — priority_queue (max-heap by default) —
std::priority_queue<int> pq;
pq.push(3); pq.push(7); pq.push(1);
pq.top();    // 7 (largest element)

```

Algorithms

```

#include <algorithm>
#include <numeric>

std::vector<int> v = {5, 3, 8, 1, 9, 2, 7};

std::sort(v.begin(), v.end());           // Sort ascending
std::sort(v.begin(), v.end(), std::greater<int>()); // Sort descending

auto it = std::find(v.begin(), v.end(), 8); // Find element
if (it != v.end()) std::cout << "Found at index " << (it - v.begin());

bool found = std::binary_search(v.begin(), v.end(), 5); // O(log n), must be sorted

int minVal = *std::min_element(v.begin(), v.end());
int maxVal = *std::max_element(v.begin(), v.end());
int sum     = std::accumulate(v.begin(), v.end(), 0);

```

```
std::reverse(v.begin(), v.end());

// Count elements matching a condition
int count = std::count_if(v.begin(), v.end(), [](int x) { return x > 5; });

// Remove duplicates (must be sorted first)
std::sort(v.begin(), v.end());
v.erase(std::unique(v.begin(), v.end()), v.end());
```

Chapter 17: Modern C++ (C++11/14/17/20)

auto Type Inference

```
auto x = 42;           // int
auto y = 3.14;         // double
auto s = std::string("hi"); // std::string
auto it = myMap.begin(); // std::map<K,V>::iterator (saves typing!)

// Use auto when the type is obvious or very long
// Don't use auto when it hurts readability
```

Lambda Functions (Anonymous Functions)

```
// Lambda syntax: [capture](parameters) -> return_type { body }

auto add = [](int a, int b) { return a + b; };
int result = add(3, 4); // 7

// Capture variables from outer scope
int threshold = 5;
auto is_above = [threshold](int x) { return x > threshold; };

// Use with STL algorithms
std::vector<int> nums = {1, 5, 3, 8, 2, 9};
std::sort(nums.begin(), nums.end(), [](int a, int b) {
    return a > b; // Sort descending
});

int count = std::count_if(nums.begin(), nums.end(), [](int x) {
    return x % 2 == 0; // Count even numbers
});
```

Smart Pointers (Automatic Memory Management)

```

#include <memory>

// unique_ptr: Sole ownership. Cannot be copied, only moved.
// When it goes out of scope, the memory is automatically freed.
{
    auto motor = std::make_unique<Motor>();
    motor->start();
    motor->set_speed(50);
} // Motor destructor runs here automatically. No memory leak possible!

// shared_ptr: Shared ownership. Reference counted.
// Memory freed when the last shared_ptr is destroyed.
auto sensor = std::make_shared<TemperatureSensor>(0x48);
auto backup = sensor; // Both point to the same object. ref_count = 2.
backup.reset();       // ref_count = 1
sensor.reset();       // ref_count = 0 → object destroyed

// weak_ptr: Non-owning observer. Doesn't prevent destruction.
std::weak_ptr<TemperatureSensor> observer = sensor;
if (auto locked = observer.lock()) {
    // Object still exists, safe to use
    locked->read();
}

```

Rule of Three/Five/Zero: In modern C++, prefer smart pointers over raw `new/delete`. This eliminates memory leaks, double-frees, and use-after-free bugs by design.

Move Semantics (C++11)

```

// Problem: Returning a large vector from a function used to COPY all data
std::vector<int> generate_data() {
    std::vector<int> result(1000000);
    // ... fill with data ...
    return result; // In C++11: MOVED, not copied. Zero overhead.
}

// std::move explicitly transfers ownership
std::string a = "Hello, World!";
std::string b = std::move(a); // b now owns the data. a is empty ("").
// No bytes were copied – just pointer/size swapped internally.

```

Chapter 18: Error Handling Patterns

C-Style Error Handling

```

// Pattern 1: Return error codes
typedef enum { OK = 0, ERR_INVALID, ERR_TIMEOUT, ERR_OVERFLOW } ErrorCode;

```

```

ErrorCode uart_send(const char *data, int len) {
    if (data == NULL) return ERR_INVALID;
    if (len > MAX_LEN) return ERR_OVERFLOW;

    for (int i = 0; i < len; i++) {
        int timeout = 1000;
        while (!(USART2->SR & TXE) && --timeout);
        if (timeout == 0) return ERR_TIMEOUT;
        USART2->DR = data[i];
    }
    return OK;
}

// Caller must check:
ErrorCode err = uart_send(msg, len);
if (err != OK) {
    printf("UART send failed: %d\n", err);
}

// Pattern 2: Return value through pointer, error through return
int read_sensor(float *out_temperature) {
    uint8_t raw = i2c_read_reg(0x48, 0x00);
    if (raw == 0xFF) return -1; // Error
    *out_temperature = raw * 0.0625f;
    return 0; // Success
}

```

C++ Exception Handling

```

#include <stdexcept>

float divide(float a, float b) {
    if (b == 0.0f) {
        throw std::invalid_argument("Division by zero");
    }
    return a / b;
}

try {
    float result = divide(10.0f, 0.0f);
    std::cout << result << std::endl;
} catch (const std::invalid_argument &e) {
    std::cerr << "Error: " << e.what() << std::endl;
} catch (const std::exception &e) {
    std::cerr << "Unexpected error: " << e.what() << std::endl;
}

```

When to use which:

- **Embedded (C):** Use error codes. Exceptions have overhead (stack unwinding, RTTI) and are unpredictable in timing — forbidden in real-time systems.
- **Desktop/Server (C++):** Exceptions are fine. They simplify error handling and prevent forgetting to check return codes.

PART IV: ENGINEERING PRACTICES

Chapter 19: Writing Clean Code

Naming Conventions

```
// Variables: snake_case, descriptive
int sensor_temperature;    // GOOD
int st;                   // BAD: what is st?
int temp;                 // BAD: temperature? temporary?

// Functions: verb_noun
void read_temperature(void); // GOOD
void update_display(void);  // GOOD
void rt(void);              // BAD

// Constants: UPPER_SNAKE_CASE
#define MAX_RETRY_COUNT 3
#define UART_BAUD_RATE 115200

// Types: PascalCase (or snake_case with _t suffix)
typedef struct SensorData { ... } SensorData;
typedef enum SystemState { ... } SystemState;
```

The Single Responsibility Principle

Every function should do ONE thing:

```
// BAD: This function does three things
void process(void) {
    int raw = read_adc();
    float temp = raw * 0.0625f;
    char buf[32];
    sprintf(buf, "%.2f C", temp);
    uart_send(buf);
}

// GOOD: Each function does one thing
int read_raw_temperature(void);
float convert_to_celsius(int raw);
void send_temperature_report(float temp);
```

```
void process(void) {
    int raw = read_raw_temperature();
    float temp = convert_to_celsius(raw);
    send_temperature_report(temp);
}
```

Comments

```
// BAD: States the obvious
i++; // Increment i

// BAD: Outdated comment (worse than no comment)
// Read temperature from sensor
float pressure = read_pressure(); // Comment says temp, code reads pressure!

// GOOD: Explains WHY, not WHAT
// Multiply by 0.0625 because the TMP102 sensor returns temperature
// as a 12-bit value in units of 0.0625°C per LSB
float temp = raw * 0.0625f;

// GOOD: Warns about non-obvious behavior
// CAUTION: This function disables interrupts for ~50µs.
// Do not call from time-critical ISR context.
void flash_write_page(uint32_t addr, uint8_t *data);
```

Chapter 20: Testing and Debugging

printf Debugging (The Fastest Method)

```
void process_data(int *data, int len) {
    printf("[DEBUG] process_data called: len=%d\n", len);
    for (int i = 0; i < len; i++) {
        printf("[DEBUG] data[%d] = %d\n", i, data[i]);
        data[i] = transform(data[i]);
        printf("[DEBUG] after transform: data[%d] = %d\n", i, data[i]);
    }
    printf("[DEBUG] process_data complete\n");
}
```

Assert (Catch Bugs Early)

```
#include <assert.h>

void set_motor_speed(int speed) {
    assert(speed >= 0 && speed <= 100); // Crash immediately if violated
```

```
// In release builds: compile with -DNDEBUG to disable all asserts
motor_pwm_set(speed);
}
```

Unit Testing in C (Minimal Framework)

```
#include <stdio.h>
#include <assert.h>

// Function to test
int add(int a, int b) { return a + b; }

// Test cases
void test_add() {
    assert(add(2, 3) == 5);
    assert(add(-1, 1) == 0);
    assert(add(0, 0) == 0);
    assert(add(-5, -3) == -8);
    assert(add(2147483647, 0) == 2147483647); // Max int
    printf("[PASS] test_add\n");
}

int main() {
    test_add();
    printf("\nAll tests passed!\n");
    return 0;
}
```

Using GDB (Command-Line Debugger)

```
# Compile with debug symbols
gcc -g -O0 program.c -o program

# Launch GDB
gdb ./program

# Essential GDB commands:
(gdb) break main           # Set breakpoint at main()
(gdb) break file.c:42      # Set breakpoint at specific line
(gdb) run                  # Start execution
(gdb) next                 # Step over (execute one line)
(gdb) step                 # Step into (enter function calls)
(gdb) print variable      # Print variable value
(gdb) print *array@10     # Print 10 elements of array
(gdb) print/x value       # Print in hexadecimal
(gdb) backtrace            # Show call stack
(gdb) watch variable      # Break when variable changes
(gdb) continue            # Resume execution
```

```
(gdb) info locals      # Show all local variables
(gdb) quit             # Exit GDB
```

Valgrind (Memory Error Detector — Linux)

```
gcc -g program.c -o program
valgrind --leak-check=full ./program

# Output example:
# ==12345== 40 bytes in 1 blocks are definitely lost
# ==12345==    at malloc (in libcmalloc.so)
# ==12345==    by main (program.c:15)
# ==12345==
# ==12345== LEAK SUMMARY:
# ==12345==    definitely lost: 40 bytes in 1 blocks
```

Chapter 21: Where to Practice

Online Judges and Problem Sets

Platform	Focus	Difficulty Range	Best For
LeetCode	Algorithms, data structures	Easy → Hard	Interview prep, daily practice
HackerRank	C/C++ specific challenges	Easy → Hard	Learning language features
Codeforces	Competitive programming	Beginner → Expert	Speed, algorithmic thinking
Project Euler	Mathematical programming	Easy → Extremely Hard	Math + code intersection
Exercism	Language learning tracks	Beginner → Intermediate	Clean code, mentoring
Advent of Code	Daily puzzles (December)	Easy → Hard	Fun, community-driven

Project Ideas (Sorted by Difficulty)

#	Project	Concepts Practiced
1	CLI Calculator	Input parsing, switch, operators
2	Number guessing game	Random numbers, loops, conditionals
3	To-do list (file-backed)	File I/O, structs, arrays
4	Simple text editor	String manipulation, cursor movement

#	Project	Concepts Practiced
5	Linked list library	Pointers, dynamic memory, generics
6	Hash table implementation	Hash functions, collision resolution
7	Markdown to HTML converter	String parsing, state machines
8	Simple HTTP server	Sockets, networking, concurrency
9	Custom malloc implementation	Memory management, alignment
10	Mini database (B-Tree based)	File I/O, B-Trees, serialization
11	Compiler for a tiny language	Lexing, parsing, code generation
12	Operating system kernel	Everything: the ultimate project

Appendix A: C/C++ Quick Reference Card

printf Format Specifiers

Specifier	Type	Example Output
%d / %i	int (decimal)	-42
%u	unsigned int	42
%x / %X	unsigned hex	2a / 2A
%o	unsigned octal	52
%f	float/double	3.141593
%e / %E	Scientific notation	3.141593e+00
%c	char	A
%s	string (char*)	Hello
%p	pointer	0x7ffd1234
%ld	long int	
%lld	long long int	
%zu	size_t	
%%	Literal %	%
%08X	Hex, zero-padded to 8	0000002A
%.2f	Float, 2 decimal places	3.14
%-20s	String, left-aligned, 20 wide	Hello

Header Files Quick Reference

Header	Key Contents
<stdio.h>	printf, scanf, fopen, fclose, fgets, fprintf
<stdlib.h>	malloc, free, calloc, realloc, atoi, atof, exit, qsort
<string.h>	strlen, strcmp, strcpy, strcat, memcpy, memset, strstr
<stdint.h>	uint8_t, int16_t, uint32_t, int64_t
<stdbool.h>	bool, true, false (C99)
<math.h>	sin, cos, sqrt, pow, fabs, ceil, floor, log
<ctype.h>	isalpha, isdigit, toupper, tolower
<assert.h>	assert()
<limits.h>	INT_MAX, INT_MIN, UINT_MAX, CHAR_BIT
<float.h>	FLT_MAX, FLT_MIN, FLT_EPSILON, DBL_MAX
<errno.h>	errno, perror()
<time.h>	time, clock, difftime, strftime
<signal.h>	signal, raise, SIGINT, SIGSEGV

Compilation Cheat Sheet

```
# C
gcc -Wall -Wextra -Werror -O2 -g main.c -o program
gcc -std=c11 -Wall main.c -lm -o program    # Link math library

# C++
g++ -Wall -Wextra -Werror -O2 -g -std=c++17 main.cpp -o program

# Compile only (no linking)
gcc -c module.c -o module.o

# Link multiple object files
gcc main.o module.o utils.o -o program

# Generate assembly
gcc -S main.c -o main.s

# Preprocess only (see macro expansion)
gcc -E main.c -o main.i

# Static analysis
cppcheck --enable=all main.c
```

You now have a complete reference for C and C++ programming. This document covers everything from thinking like a programmer, through the language mechanics, data structures, algorithms, memory management, OOP, modern C++ features, to engineering best practices. Refer back to specific chapters as you need them — and most importantly, **write code every day.**